

第一部分：理论知识

1.1 为什么需要软件包管理？

生活化类比：手机应用商店

想象一下你在手机上安装应用的场景：

传统方式（没有应用商店）	现代方式（应用商店）
到处搜索下载链接	打开应用商店搜索
担心下载到病毒	官方审核，安全可靠
手动查找更新	自动提示更新
卸载残留垃圾文件	一键完整卸载
依赖问题自己解决	自动安装依赖

Linux 的软件包管理器就像是命令行版的"应用商店"！

软件包管理解决的核心问题

软件包管理器解决的问题		
	统一的安装格式	- 不用关心软件怎么编译
	依赖关系处理	- 自动安装需要的其他软件包
	版本管理	- 轻松升级、降级、回滚
	完整卸载	- 干净移除，不留垃圾
	完整性验证	- 确保软件包没被篡改
	软件查询	- 知道装了什么，文件在哪

1.2 RPM 软件包格式

RPM 包命名格式

`nginx-1.20.1-1.el8.x86_64.rpm`

|

|

|

|

|

|

└─ 扩展名

└─ 架构: x86_64(64位)、i686(32位)、noarch(通用)

└─ 适用系统: el8=RHEL8/CentOS8, el7=RHEL7/CentOS7

└─ 发行版本号 (打包次数)

└─ 软件版本号

└─ 软件名称

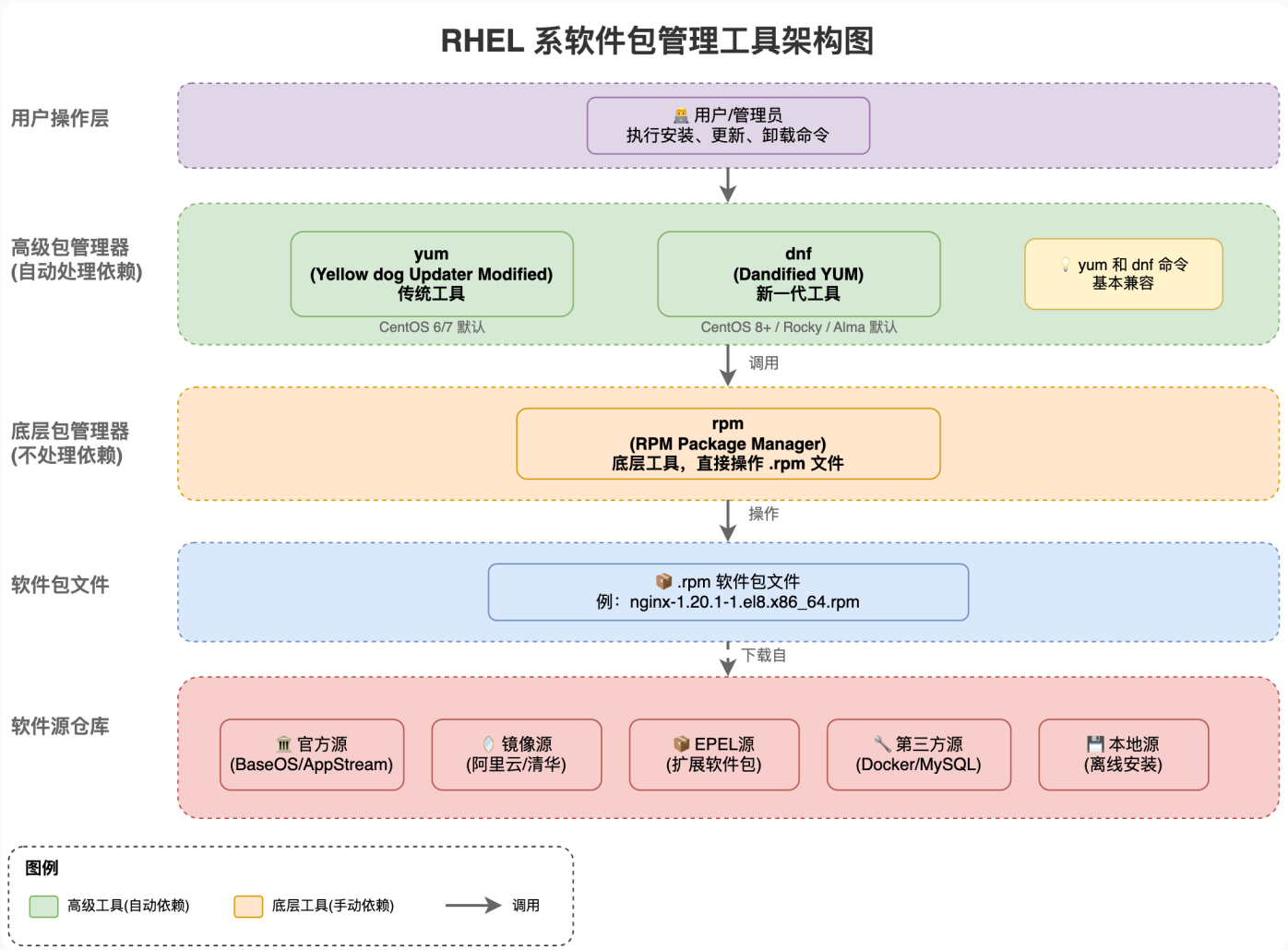
常见架构说明

架构	说明
x86_64	64 位 Intel/AMD 处理器
i686 / i386	32 位处理器
noarch	与架构无关（脚本、配置文件等）
aarch64	ARM 64 位处理器

系统版本代号

代号	对应系统
el7	RHEL 7 / CentOS 7
el8	RHEL 8 / CentOS 8 / Rocky Linux 8 / AlmaLinux 8
el9	RHEL 9 / Rocky Linux 9 / AlmaLinux 9

1.3 软件包管理工具层次



工具对比

特性	rpm	yum	dnf
依赖处理	❌ 不处理	✅ 自动处理	✅ 自动处理
软件源支持	❌ 不支持	✅ 支持	✅ 支持
性能	快	较慢	快
内存占用	低	高	中等
默认系统	所有 RHEL 系	CentOS 6/7	CentOS 8+/Rocky

1.4 软件源（Repository）详解

生活化类比：购物渠道

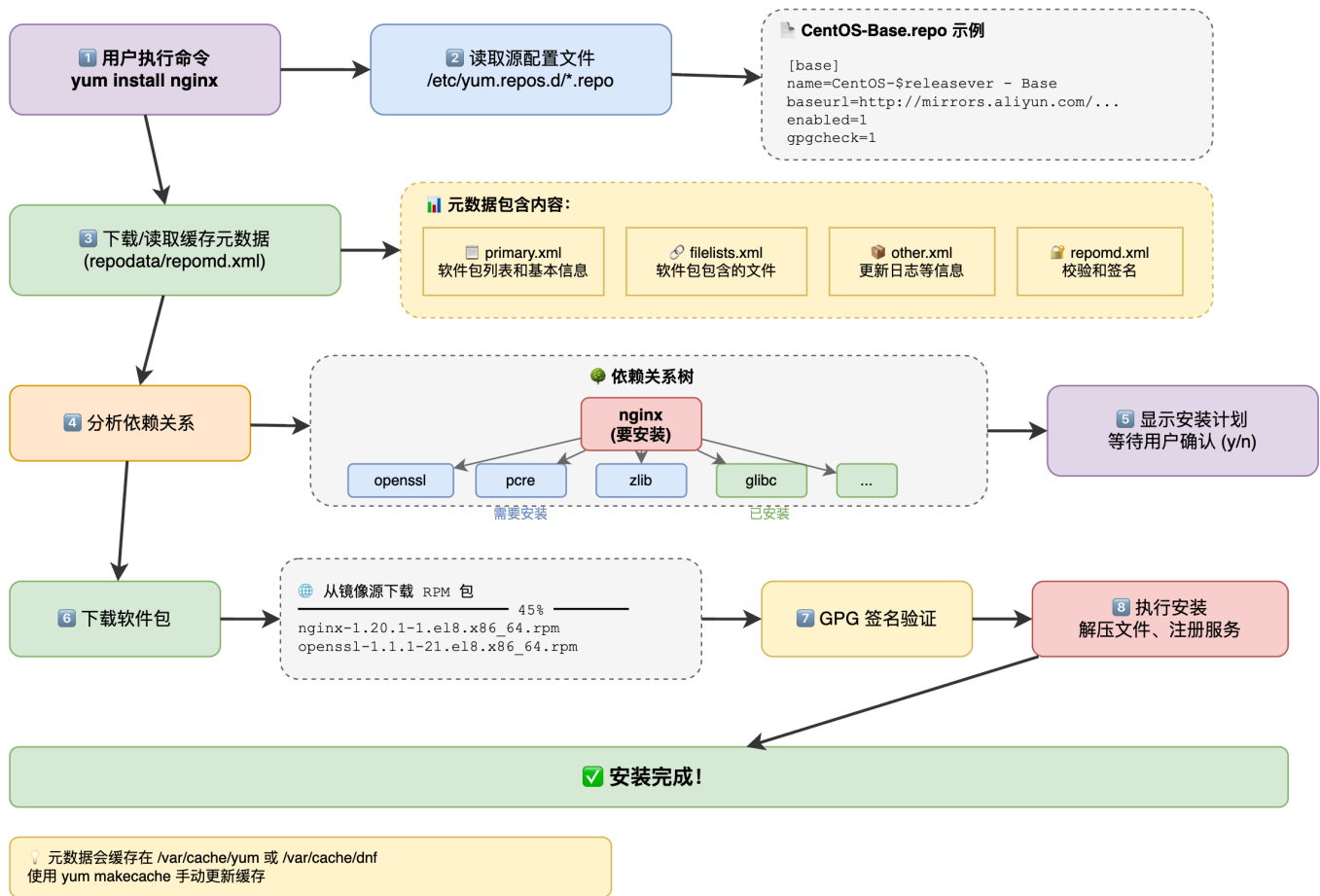
软件源类型	类比	特点
官方源	品牌官方旗舰店	最权威，但可能速度慢
镜像源	官方授权代理商	内容相同，本地化服务更快
EPEL源	官方合作伙伴店	扩展商品，质量有保证
第三方源	专卖店	特定软件的最新版本
本地源	自家仓库	离线可用，完全自主

源配置文件位置

```
/etc/yum.repos.d/           # 源配置目录
├-- CentOS-Base.repo        # 基础源
├-- CentOS-AppStream.repo   # 应用流源（CentOS 8+）
├-- epel.repo               # EPEL 扩展源
└-- docker-ce.repo         # Docker 第三方源
```

配图 2：软件源工作流程图

YUM/DNF 软件安装工作流程



1.5 依赖关系详解

什么是依赖？

生活类比：做一道菜需要的食材

要做 "番茄炒蛋" 需要：

- └─ 番茄（必需）
- └─ 鸡蛋（必需）
- └─ 盐（必需）
- └─ 油（必需）
- └─ 葱花（可选）

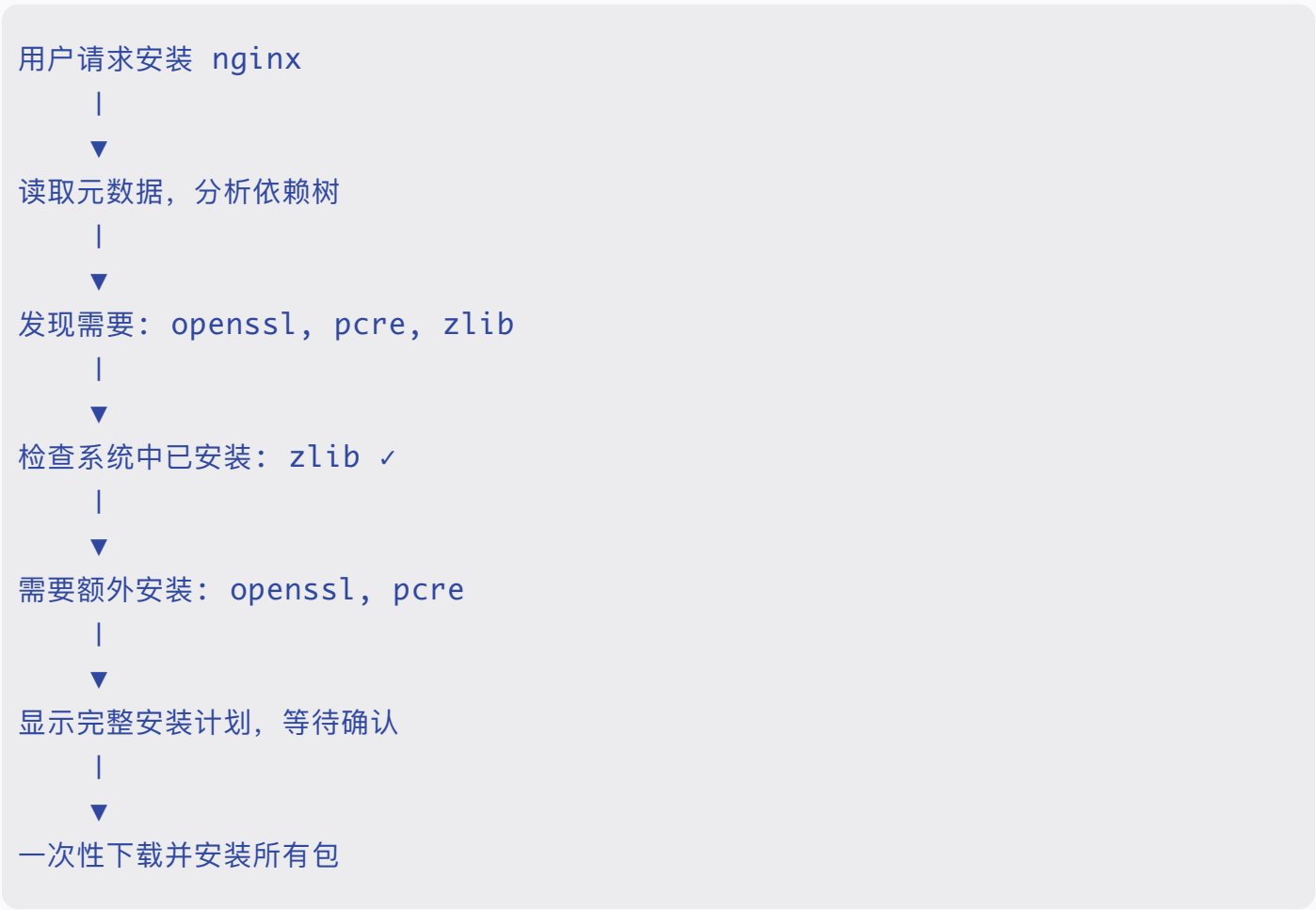
类似地，安装 nginx 需要：

- └─ openssl（加密库）
- └─ pcre（正则表达式库）
- └─ zlib（压缩库）

依赖问题类型

问题类型	说明	解决方案
缺少依赖	需要的包没有安装	使用 yum/dnf 自动解决
版本冲突	需要的版本与现有冲突	升级/降级相关包
循环依赖	A 依赖 B，B 又依赖 A	同时安装两个包
依赖地狱	复杂的依赖链难以满足	使用容器或虚拟环境

yum/dnf 如何解决依赖



第二部分：实践操作

2.1 实验环境确认

检查系统版本

```
# 查看系统版本
cat /etc/os-release

# 或者查看 redhat-release
cat /etc/redhat-release

# 查看系统架构
uname -m

# 检查包管理器版本
yum --version
# 或
dnf --version
```

2.2 实验 1：YUM/DNF 基础操作

实验目标

掌握 yum/dnf 的基本使用方法。

 **提示：**在 CentOS 8+、Rocky Linux、AlmaLinux 中，`yum` 是 `dnf` 的软链接，两者命令完全兼容。本课程统一使用 `yum` 命令演示。

实验步骤

步骤 1：查看软件源配置

```
# 查看已启用的软件源
yum repolist

# 查看所有源（包括禁用的）
yum repolist all

# 查看源配置文件目录
ls -la /etc/yum.repos.d/
```

步骤 2：搜索软件包

```
# 搜索软件包（模糊匹配名称和描述）
yum search nginx

# 列出可更新的软件包
yum list updates
```

关键输出解读：

```
[root@localhost ~]# yum info nginx
Name           : nginx
Version        : 1.14.1
Release        : 9.module+el8.0.0+4108+af250afe
Architecture   : x86_64
Size           : 570 k
Source         : nginx-1.14.1-9.module+el8.0.0+4108+af250afe.src.rpm
Repository     : appstream          # 来自哪个源
Summary        : A high performance web server and reverse proxy server
License        : BSD
Description    : Nginx is a web server and a reverse proxy server...
```

步骤 3：安装软件包

```
# 安装单个软件包
yum install -y httpd
```

```
# 安装多个软件包
yum install -y vim wget curl tree

# 安装软件包组（开发工具）
yum groupinstall -y "Development Tools" # 安装软件组

# 安装指定版本（如果有多个版本可用）
yum list nginx --showduplicates          # 查看所有可用版本
yum install -y nginx-1.14.1

# 从本地 rpm 文件安装（自动解决依赖）
yum localinstall /tmp/packages/httpd-2.4.*.rpm
```

步骤 4：查看已安装软件

```
# 查看软件包是否已安装
yum list installed | grep httpd
```

步骤 5：更新软件包

```
# 检查可用更新
yum check-update

# 更新单个软件包
yum update -y httpd

# 更新所有软件包（生产环境谨慎操作）
yum update -y

# 只更新安全补丁
yum update --security

# 查看更新历史
yum history
yum history info 3    # 查看第3次操作的详情
```

步骤 6：卸载软件包

```
# 卸载软件包（保留依赖）
yum remove httpd

# 卸载软件包组
yum groupremove "Development Tools"
```

步骤 7：缓存管理

```
# 清理软件包缓存
yum clean packages

# 清理所有缓存
yum clean all

# 重建缓存
yum makecache
```

RPM底层操作

查询 RPM 包信息

# 查询已安装的包	# 查询是否安装
<code>rpm -qa grep wget</code>	# 查询包详细信息
<code>rpm -qi wget</code>	# 查询包安装的文件列表
<code>rpm -ql wget</code>	# 查询包的配置文件
<code>rpm -qc wget</code>	# 查询包的文档文件
<code>rpm -qd wget</code>	



YUM/DNF 命令速查表

操作	命令	说明
搜索软件包	<code>yum search <关键词></code>	模糊搜索
查看包信息	<code>yum info <包名></code>	显示详细信息
安装软件包	<code>yum install -y <包名></code>	-y 自动确认
卸载软件包	<code>yum remove <包名></code>	保留依赖
自动卸载	<code>yum autoremove <包名></code>	清理无用依赖
更新软件包	<code>yum update -y <包名></code>	更新指定包
更新所有	<code>yum update -y</code>	更新全部
检查更新	<code>yum check-update</code>	查看可更新的包
列出已安装	<code>yum list installed</code>	列出所有已安装包
查看软件源	<code>yum repolist</code>	查看启用的源
清理缓存	<code>yum clean all</code>	清理所有缓存
重建缓存	<code>yum makecache</code>	更新元数据缓存
查看历史	<code>yum history</code>	查看操作历史
回滚操作	<code>yum history undo <ID></code>	撤销指定操作

2.3 实验 2：配置软件源

实验目标

掌握软件源的配置和管理，学会使用国内镜像源加速下载。

实验步骤

步骤 1：备份原有配置

```
# 创建备份目录
mkdir -p /etc/yum.repos.d/backup

# 备份所有 repo 文件
cp /etc/yum.repos.d/*.repo /etc/yum.repos.d/backup/

# 验证备份
ls -la /etc/yum.repos.d/backup/
```

步骤 2：配置镜像源

系统软件源的地址：

阿里云：🔗<https://developer.aliyun.com/mirror/>

南京大学：🔗<https://mirrors.nju.edu.cn/>

华为：🔗<https://mirrors.huaweicloud.com/home>

找到对应的系统或软件，然后按照官网操作即可

步骤 3：添加 MySQL 官方源

```
# 下载 MySQL 源配置包
# CentOS 7
yum install -y https://dev.mysql.com/get/mysql80-community-release-
el7-7.noarch.rpm

# CentOS 8 / Rocky 8
yum install -y https://dev.mysql.com/get/mysql80-community-release-
el8-4.noarch.rpm

# 查看可用的 MySQL 版本
yum repolist all | grep mysql
```

```
# 禁用 MySQL 8.0, 启用 MySQL 5.7 (如需要)
yum-config-manager --disable mysql80-community
yum-config-manager --enable mysql57-community

# 安装 MySQL
yum install -y mysql-community-server
```

步骤 4：禁用/启用软件源

```
# 临时禁用某个源进行安装
yum --disablerepo=epel install <package>

# 临时启用某个源进行安装
yum --enablerepo=epel-testing install <package>

# 永久禁用源（修改配置文件）
yum-config-manager --disable epel

# 永久启用源
yum-config-manager --enable epel

# 手动编辑配置文件
vim /etc/yum.repos.d/epel.repo
# 将 enabled=1 改为 enabled=0
```

2.4 实验 3：源码编译安装

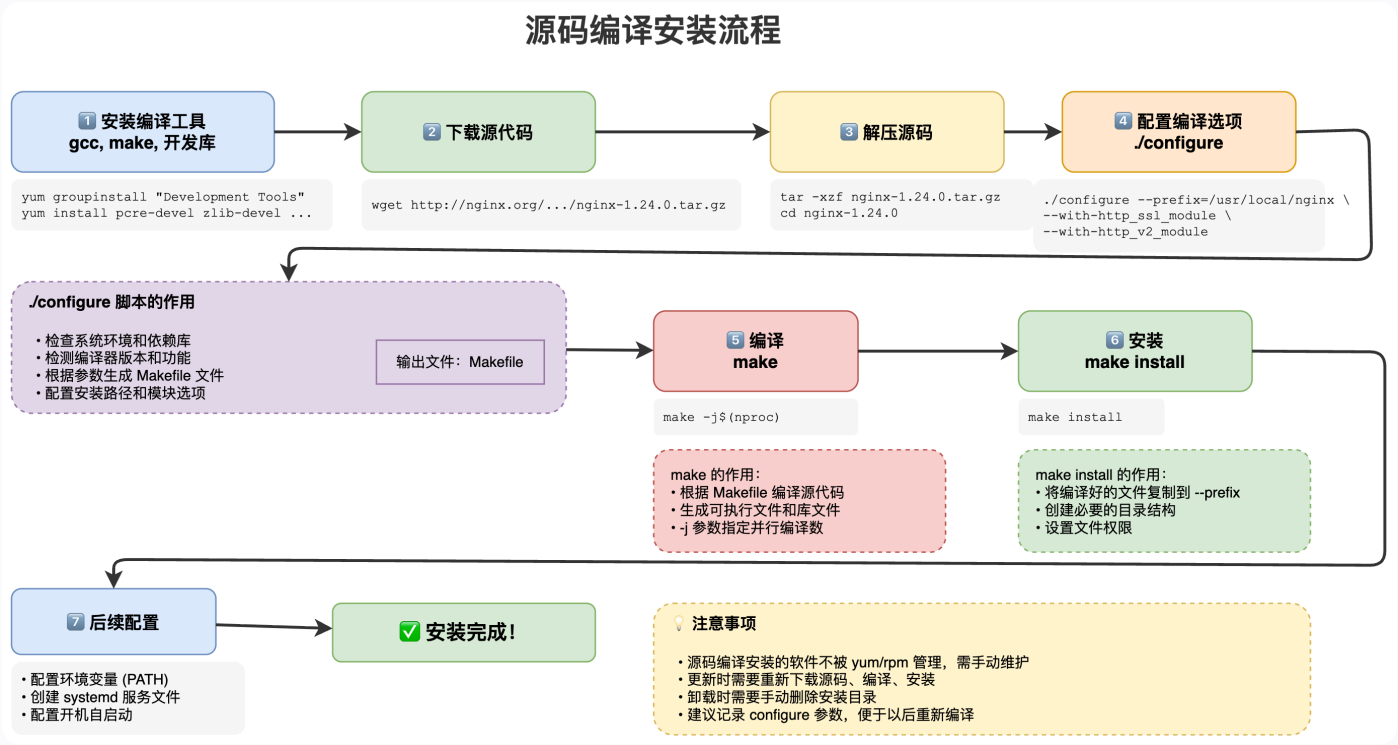
实验目标

掌握从源代码编译安装软件的方法，理解软件构建过程。

为什么需要源码编译？

场景	说明
版本需求	需要比软件源更新/更旧的版本
定制功能	需要启用/禁用特定模块和功能
性能优化	针对特定 CPU 架构优化编译
安全加固	使用特定的安全编译选项
学习目的	理解软件构建过程

配图 3：源码编译流程图



实验步骤：编译安装 Nginx

步骤 1：安装编译工具和依赖

```
# 安装开发工具组
yum groupinstall -y "Development Tools"
```

```
# 安装 Nginx 编译所需的依赖库
yum install -y \
    pcre-devel \      # 正则表达式库
    zlib-devel \      # 压缩库
    openssl-devel \   # SSL/TLS 加密库
    libxml2-devel \   # XML 解析库
    libxslt-devel \   # XSLT 转换库
    gd-devel \        # 图像处理库
    GeoIP-devel \      # IP 地理位置库

# 验证 gcc 已安装
gcc --version
make --version
```

步骤 2：下载源代码

```
# 创建编译目录
mkdir -p /opt/src
cd /opt/src

# 下载 Nginx 源码（选择稳定版）
wget http://nginx.org/download/nginx-1.24.0.tar.gz

# 验证下载
ls -lh nginx-1.24.0.tar.gz

# 解压
tar -xzf nginx-1.24.0.tar.gz

# 进入源码目录
cd nginx-1.24.0
ls -la
```

步骤 3：配置编译选项

```
# 查看所有可用配置选项
./configure --help | less
```

```
# 常用配置（生产环境推荐）
./configure \
    --prefix=/usr/local/nginx \
    --user=nginx \
    --group=nginx \
    --with-http_ssl_module \
    --with-http_v2_module \
    --with-http_realip_module \
    --with-http_gzip_static_module \
    --with-http_stub_status_module \
    --with-pcre \
    --with-stream \
    --with-stream_ssl_module

# 配置完成后查看摘要信息
```

配置参数说明：

参数	说明
<code>--prefix=/usr/local/nginx</code>	安装目录
<code>--user=nginx</code>	运行用户
<code>--group=nginx</code>	运行用户组
<code>--with-http_ssl_module</code>	启用 HTTPS 支持
<code>--with-http_v2_module</code>	启用 HTTP/2 支持
<code>--with-http_realip_module</code>	获取真实客户端 IP
<code>--with-http_gzip_static_module</code>	静态 gzip 压缩
<code>--with-http_stub_status_module</code>	状态监控页面
<code>--with-stream</code>	启用 TCP/UDP 代理
<code>--without-http_xxx_module</code>	禁用某个模块

步骤 4：编译

```
# 编译（使用所有 CPU 核心）
make -j$(nproc)

# 或指定并行数
make -j4

# 编译需要几分钟，成功后会显示：
# make[1]: Leaving directory '/opt/src/nginx-1.24.0'
```

步骤 5：安装

```
# 创建运行用户（不创建家目录，不允许登录）
useradd -r -s /sbin/nologin nginx

# 安装
make install

# 查看安装结果
ls -la /usr/local/nginx/
# conf/      - 配置文件目录
# html/      - 默认网站目录
# logs/      - 日志目录
# sbin/      - 可执行文件目录
```

步骤 6：配置环境变量

```
# 添加到 PATH
cat > /etc/profile.d/nginx.sh << 'EOF'
export PATH=/usr/local/nginx/sbin:$PATH
EOF

# 使配置生效
source /etc/profile.d/nginx.sh

# 验证
which nginx
nginx -v
nginx -V      # 显示编译参数
```

步骤 7：创建 Systemd 服务

```
# 创建服务文件
cat > /etc/systemd/system/nginx.service << 'EOF'
[Unit]
Description=The NGINX HTTP and reverse proxy server
Documentation=http://nginx.org/en/docs/
After=syslog.target network-online.target remote-fs.target nss-
lookup.target
Wants=network-online.target

[Service]
Type=forking
PIDFile=/usr/local/nginx/logs/nginx.pid
ExecStartPre=/usr/local/nginx/sbin/nginx -t
ExecStart=/usr/local/nginx/sbin/nginx
ExecReload=/bin/kill -s HUP $MAINPID
ExecStop=/bin/kill -s QUIT $MAINPID
PrivateTmp=true

[Install]
WantedBy=multi-user.target
EOF

# 重新加载 systemd
systemctl daemon-reload
```

```
# 启动 Nginx
systemctl start nginx

# 设置开机自启
systemctl enable nginx

# 查看状态
systemctl status nginx
```

步骤 8：验证安装

```
# 检查进程
ps aux | grep nginx

# 检查端口
ss -tlnp | grep :80

# 本地访问测试
curl http://localhost

# 查看版本和编译信息
nginx -V

# 测试配置文件语法
nginx -t
```

步骤 9：常用管理命令

```
# 使用 systemctl 管理
systemctl start nginx      # 启动
systemctl stop nginx       # 停止
systemctl restart nginx    # 重启
systemctl reload nginx     # 重新加载配置（不中断服务）
systemctl status nginx     # 查看状态

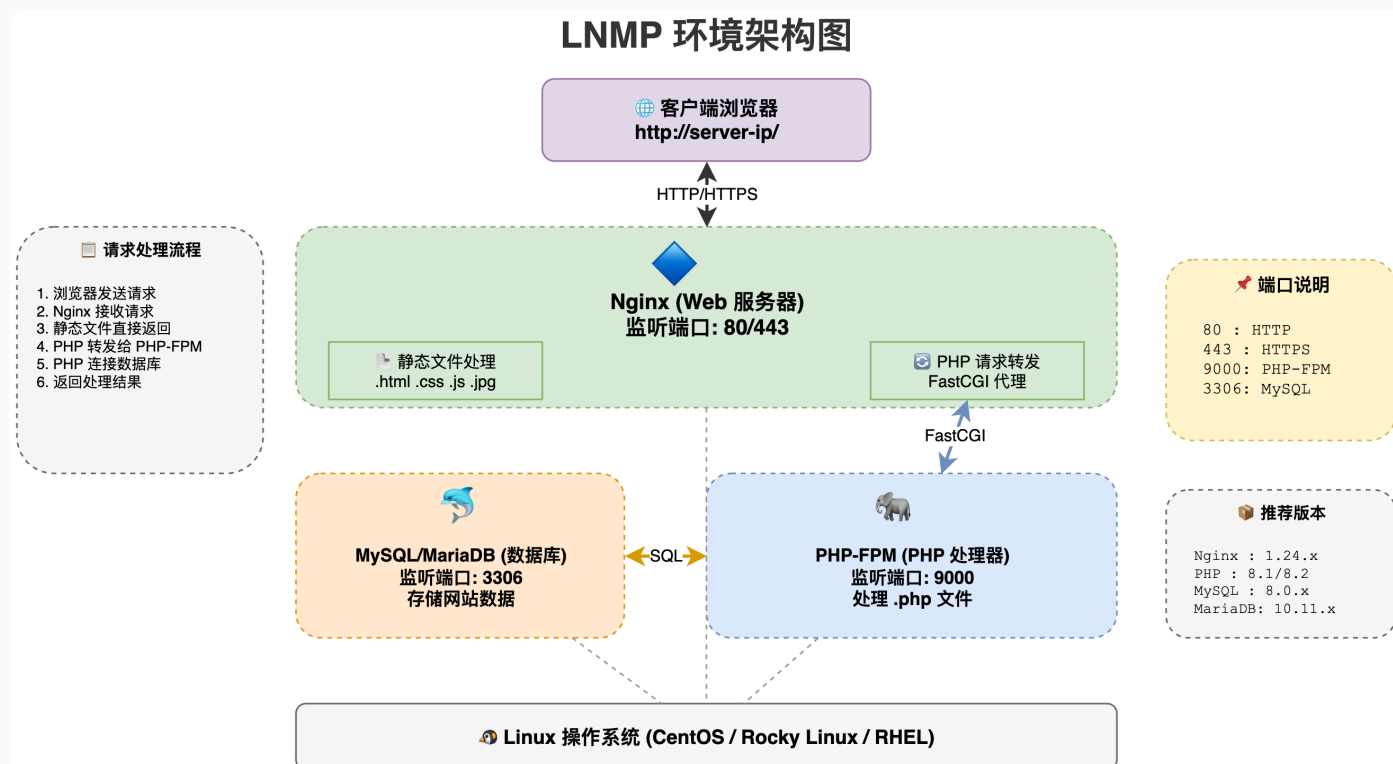
# 或直接使用 nginx 命令
nginx                      # 启动
nginx -s stop              # 快速停止
nginx -s quit              # 优雅停止（等待请求处理完）
nginx -s reload            # 重新加载配置
nginx -t                   # 测试配置
```

2.5 实验 4：综合实战 – 搭建 LNMP 环境

🎯 实验目标

综合运用所学知识，完成 LNMP（Linux + Nginx + MySQL + PHP）环境搭建。

配图 4：LNMP 架构图





实验步骤

步骤 1：配置软件源

```
# 确保已配置阿里云镜像源
# 安装 EPEL 源
# yum install -y epel-release
```

步骤 2：安装 Nginx

```
# 安装 Nginx
yum install -y nginx

# 启动并设置开机自启
systemctl start nginx
systemctl enable nginx

# 验证
systemctl status nginx
nginx -v

# 测试访问
curl -I http://localhost
```

步骤 3：安装 MySQL/MariaDB

```
# 方案1: 安装 MariaDB
yum install -y mariadb-server mariadb

# 方案2: 安装 MySQL 8.0 (需要添加 MySQL 官方源)
# yum install -y mysql-server

# 启动并设置开机自启
systemctl start mariadb
systemctl enable mariadb

# 安全初始化 (设置 root 密码等)
```

```
mysql_secure_installation
# 按提示操作：
# - 设置 root 密码
# - 删除匿名用户
# - 禁止 root 远程登录
# - 删除测试数据库
# - 重新加载权限表

# 验证
systemctl status mariadb
mysql -u root -p -e "SELECT VERSION();"
```

步骤 4：安装 PHP

```
# 安装 PHP 及常用扩展
yum install -y \
    php \
    php-fpm \
    php-mysqlnd \
    php-gd \
    php-mbstring \
    php-xml \
    php-json \
    php-opcache \
    php-curl \
    php-zip

# 启动 PHP-FPM
systemctl start php-fpm
systemctl enable php-fpm

# 验证
systemctl status php-fpm
php -v
php -m | grep mysql
```

步骤 5：配置 Nginx 支持 PHP

```
# 备份默认配置
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak

# 创建网站配置文件
cat > /etc/nginx/conf.d/default.conf << 'EOF'
server {
    listen      80;
    server_name localhost;
    root        /usr/share/nginx/html;
    index       index.php index.html index.htm;

    # 日志配置
    access_log  /var/log/nginx/access.log;
    error_log   /var/log/nginx/error.log;

    # 静态文件处理
    location / {
        try_files $uri $uri/ =404;
    }

    # PHP 文件处理
    location ~ \.php$ {
        fastcgi_pass   127.0.0.1:9000;
        fastcgi_index  index.php;
        fastcgi_param  SCRIPT_FILENAME
$document_root$fastcgi_script_name;
        include        fastcgi_params;
    }

    # 禁止访问隐藏文件
    location ~ /\. {
        deny all;
    }
}
EOF

# 修改 PHP-FPM 配置（如果需要）
# vim /etc/php-fpm.d/www.conf
# 确保 listen = 127.0.0.1:9000

# 测试 Nginx 配置
nginx -t
```

```
# 重新加载配置
systemctl reload nginx
```

步骤 6：创建测试页面

```
# 创建 PHP 信息页面
cat > /usr/share/nginx/html/info.php << 'EOF'
<?php
phpinfo();
?>
EOF

# 设置权限
chown -R nginx:nginx /usr/share/nginx/html/
chmod 644 /usr/share/nginx/html/*.php
```

步骤 7：验证 LNMP 环境

```
# 检查所有服务状态
echo "=== 服务状态 ==="
systemctl is-active nginx php-fpm mariadb

# 检查监听端口
echo "=== 监听端口 ==="
ss -tlnp | grep -E '80|443|9000|3306'

# 测试 PHP
echo "=== PHP 测试 ==="
curl -s http://localhost/info.php | head -20

# 显示版本信息
echo "=== LNMP 版本信息 ==="
echo "Nginx   : $(nginx -v 2>&1 | cut -d/ -f2)"
echo "PHP     : $(php -v | head -1)"
echo "MySQL    : $(mysql -V | awk '{print $5}' | tr -d ',')"

# 访问测试页面
```

```
echo "=== 测试页面 ==="
echo "PHP Info      : http://YOUR-IP/info.php"
echo "DB Test       : http://YOUR-IP/db_test.php"
```

2.7 实验清理

完成实验后，清理测试环境：

```
#!/bin/bash
# LNMP 环境清理脚本

echo "=== 停止服务 ==="
systemctl stop nginx php-fpm mariadb

echo "=== 禁用开机自启 ==="
systemctl disable nginx php-fpm mariadb

echo "=== 卸载软件包 ==="
yum remove -y nginx php php-fpm php-mysqlnd php-gd php-mbstring \
    php-xml php-json php-opcache php-curl php-zip \
    mariadb-server mariadb

echo "=== 清理配置和数据 ==="
rm -rf /etc/nginx
rm -rf /var/lib/mysql
rm -rf /usr/share/nginx/html/*.php

echo "=== 清理源码编译的 Nginx (如果有) ==="
rm -rf /usr/local/nginx
rm -f /etc/systemd/system/nginx.service
rm -f /etc/profile.d/nginx.sh
systemctl daemon-reload

echo "=== 删除用户 ==="
userdel -r nginx 2>/dev/null

echo "=== 清理临时文件 ==="
rm -rf /opt/src
rm -rf /tmp/rpms
```

```
rm -rf /tmp/packages
rm -rf /opt/localrepo

echo "=== 清理 yum 缓存 ==="
yum clean all

echo "=== 清理完成! ==="
```

第三部分：课程总结

命令速查表

YUM/DNF 命令

操作	命令
搜索软件包	<code>yum search <关键词></code>
查看包信息	<code>yum info <包名></code>
安装软件包	<code>yum install -y <包名></code>
卸载软件包	<code>yum remove <包名></code>
更新软件包	<code>yum update -y <包名></code>
更新所有包	<code>yum update -y</code>
列出已安装包	<code>yum list installed</code>
查看软件源	<code>yum repolist</code>
清理缓存	<code>yum clean all</code>
重建缓存	<code>yum makecache</code>
查看操作历史	<code>yum history</code>
回滚操作	<code>yum history undo <ID></code>
只下载不安装	<code>yum install --downloadonly --downloadaddir=/path <包名></code>
安装本地包	<code>yum localinstall <rpm文件></code>
安装软件组	<code>yum groupinstall "<组名>"</code>

RPM 命令

操作	命令
查询所有包	<code>rpm -qa</code>
查询包信息	<code>rpm -qi <包名></code>
查询包文件	<code>rpm -ql <包名></code>
查询配置文件	<code>rpm -qc <包名></code>

源码编译

步骤	命令
安装编译工具	<code>yum groupinstall "Development Tools"</code>
下载源码	<code>wget <URL></code>
解压	<code>tar -xzf package.tar.gz</code>
配置	<code>./configure --prefix=/usr/local/<name></code>
编译	<code>make -j\$(nproc)</code>
安装	<code>make install</code>

常见问题解答

Q1: yum 和 dnf 有什么区别?

dnf 是 yum 的下一代版本，性能更好，依赖解析更智能。在 CentOS 8+ 和 Rocky Linux 中，yum 是 dnf 的软链接，命令完全兼容。建议在新系统中使用 dnf。

Q2: 安装时提示 "No package xxx available" 怎么办?

可能原因及解决方案：

- 1. 包名拼写错误 → 使用 `yum search` 搜索正确包名
- 2. 软件源未启用 → 使用 `yum repolist all` 检查，启用相应源
- 3. 需要 EPEL 源 → 安装 `yum install epel-release`
- 4. 需要第三方源 → 添加对应的第三方源

Q3：如何回滚软件包更新？

```
# 查看操作历史
yum history

# 回滚指定操作
yum history undo <ID>

# 回滚到指定时间点
yum history rollback <ID>
```

Q4：rpm 安装提示依赖问题怎么办？

推荐使用 `yum localinstall package.rpm`，它会自动从软件源解决依赖。不建议使用 `rpm --nodeps` 强制安装。

Q5：源码编译和包管理器安装哪个更好？

各有优缺点：

方式	优点	缺点
包管理器	简单、易维护、自动更新	版本可能不是最新
源码编译	可定制、版本灵活	维护困难、更新麻烦

生产环境推荐：优先使用包管理器，除非有特殊定制需求。